REQUIREMENTS NOT MET

N/A

PROBLEMS ENCOUNTERED

1. I was unable to find ALE1 on the microcontroller.
2. I was having trouble adding setting up the null terminator for the txt data file.
3. I was confuse of the address decoding of the I/O and SRAM.
4. I didn't pay attention to the activation level of the PLD logic, resulting that the LED not showing output.

FUTURE WORK/APPLICATIONS

N/A

PRE-LAB EXERCISES

i. For each SRAM configuration within the EBI system of the ATxmega128A1U microcontroller, to which address lines do you have external, physical access? Additionally, for the SRAM 3-PORT ALE1 configuration, is it possible to have external, physical access to any address lines above A15? Why or why not?

For a 4-port non-multiplexed mode provides direct access to the full 24-bit address bus (A23-A0), while more pin-efficient modes like 3-port or 2-port multiplexed modes provide physical access up to A19.

For 3-PORT ALE1 configuration, it is possible to have external, physical access to address lines above A15 because Chip select has a port pin [3:0](A[19:16]), these pins can provide physical access to the address bus up to A19.

ii. Describe what performing full address decoding and partial address decoding signifies. Provide examples of both types for [1] an SRAM chip and [2] either an input port or output port.

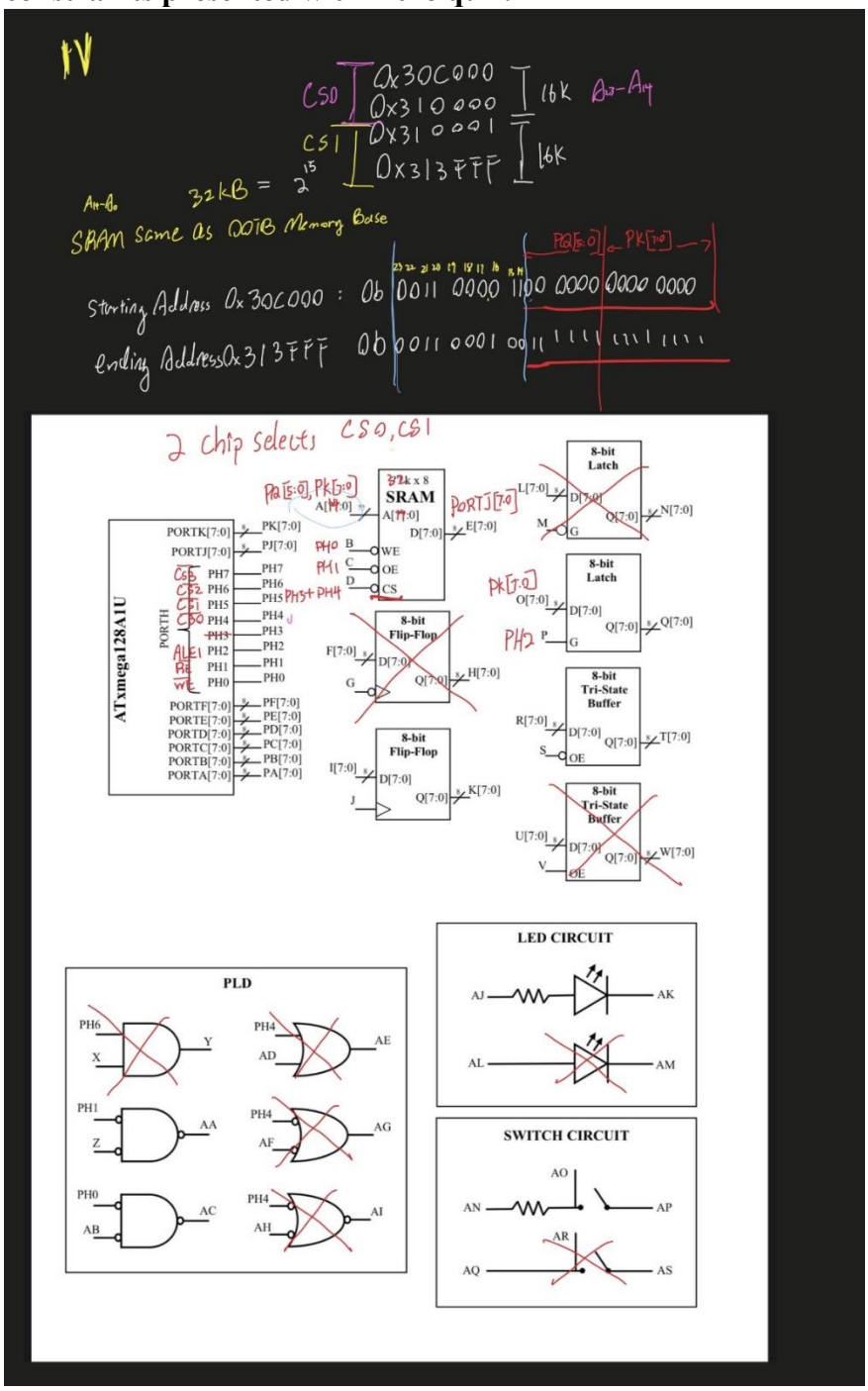
Full address decoding uses all address line to select each device, ensuring no overlapping in the memory (requires more memory). For example, 32KB SRAM mapped exactly to 0xA68000-0xA6FFFF. It will check the low address line to identify the single, specific output enable.

Partial address decoding uses only some address line and ignoring the rest. For example,
For input enable, it only check addresses line 2,1,0 and as well as the CS2. For output enable, it only check addresses 2,1,0 and data 7, 6.

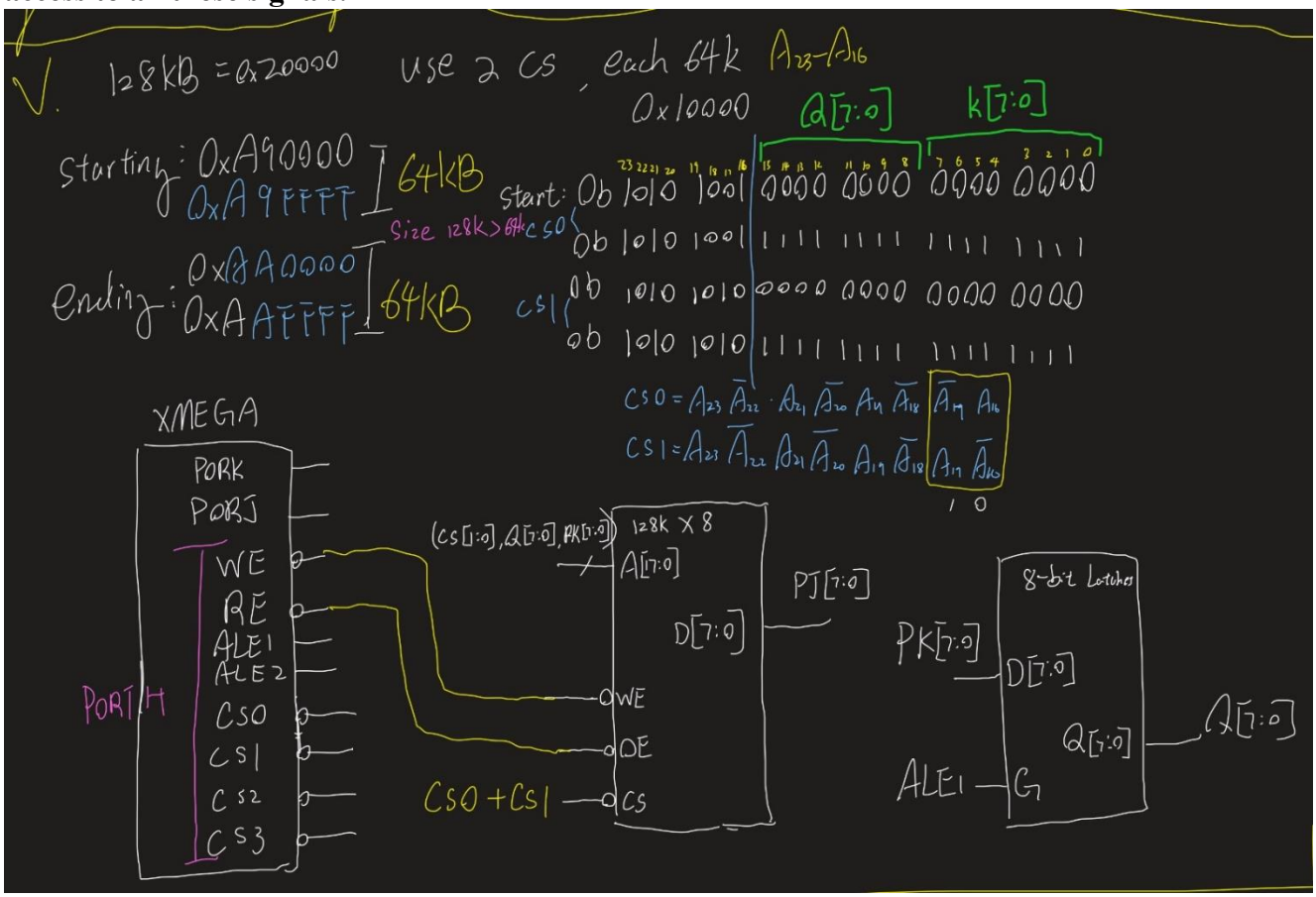
iii. In theory, how many 8-bit I/O ports could be mapped to the external data memory space of an ATxmega128A1U microcontroller, assuming that at most one port utilizes any particular address range? Explain your answer, utilizing your response from Exercise.

Since the microcontroller has 24 bits address bus, it provides 2^24 unique byte locations.

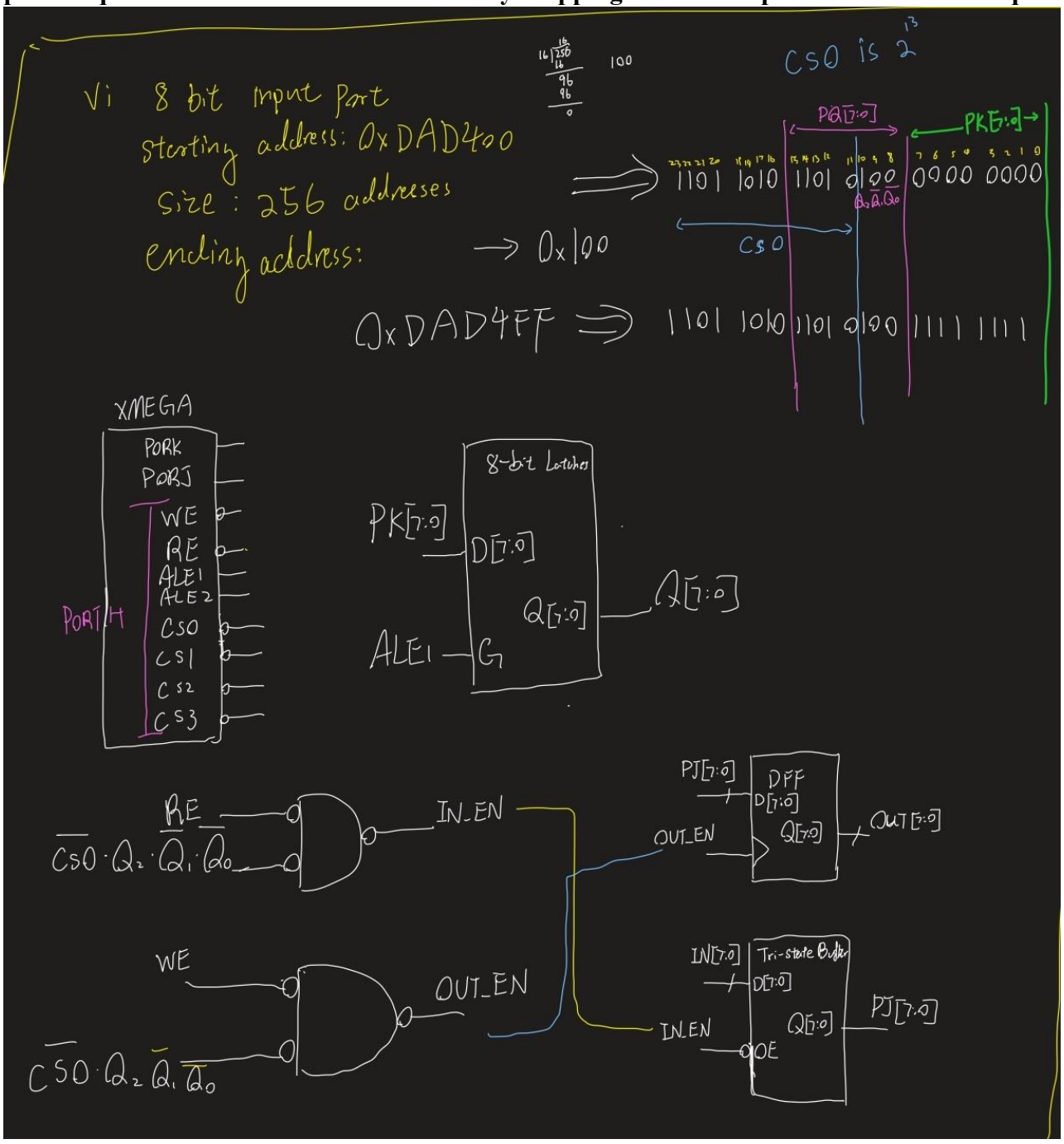
iv. Assume that an SRAM component with the same size as the one on the OOTB Memory Base has to be added to a different computing system containing an ATxmega128A1U but no OOTB Memory Base (i.e., this is not the OOTB µPAD computing system), with the first address of the SRAM starting at address 0x30 C000, instead of the address specified in the "Lab 4 – Hardware Expansion" quiz. Design, on paper, a hardware expansion for this new system, utilizing the same structure of steps laid out in this quiz. Use the same "overall" memory-mapping constraints presented within the quiz.



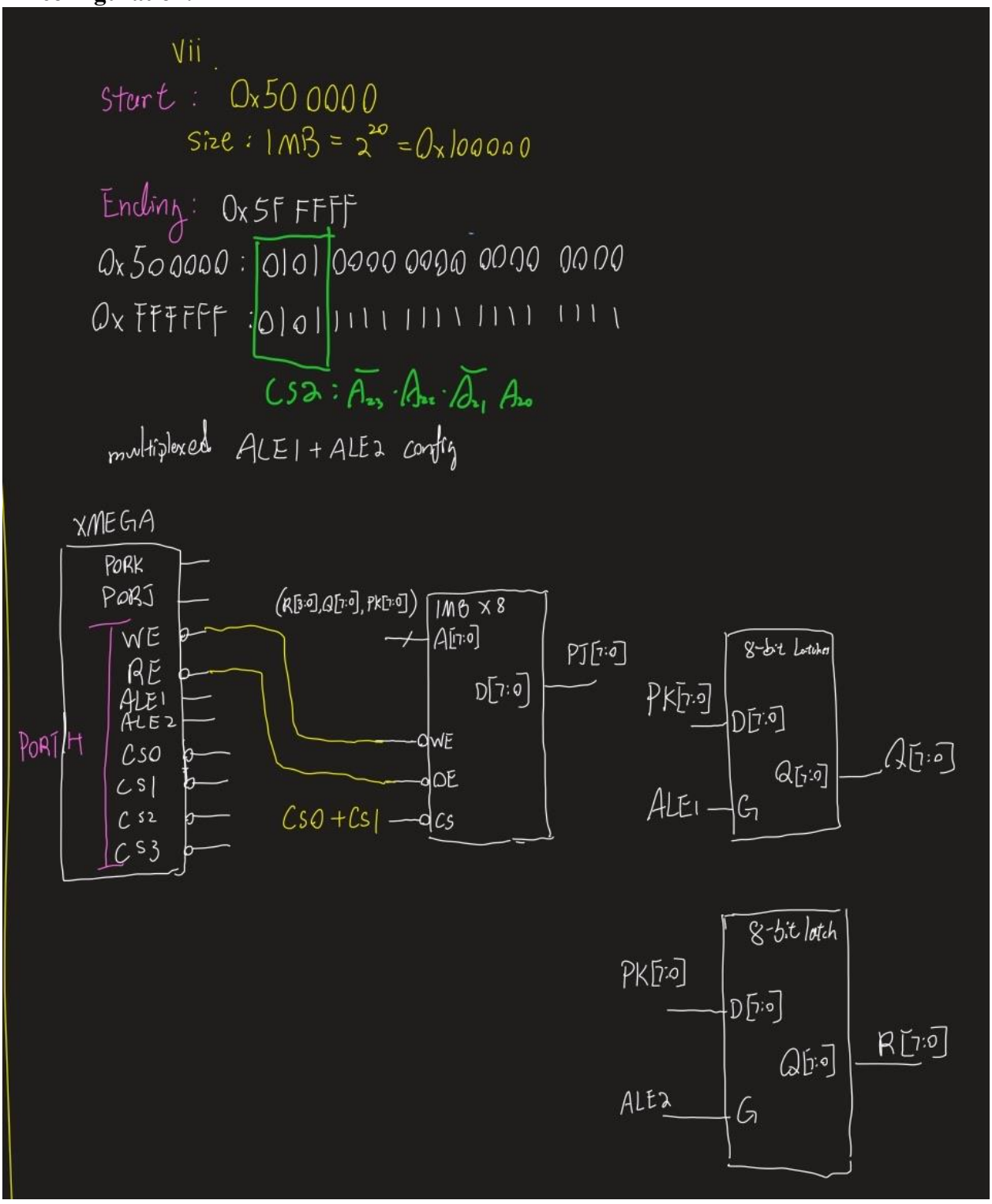
v. Assume that some external 128 KB SRAM must be fully mapped to the data memory space of the ATxmega128A1U, with the first memory location of the SRAM corresponding to data memory address 0xA9 0000. Design, on paper, a hardware expansion for this new system, utilizing the same structure of steps laid out in the relevant pre-lab quiz. Use the same "overall" memory-mapping constraints presented within this quiz. Hint: Consider how many address signals are needed to make the SRAM fully addressable and then consider how you will gain access to all these signals.



vi. Assume that some 8-bit input port should be accessible via the 256 consecutive addresses starting at 0xDA D400 within the ATxmega128A1U data memory space. Design, on paper, a hardware expansion for this port, utilizing the same structure of steps laid out in the relevant pre-lab quiz. Use the same "overall" memory-mapping constraints presented within this quiz.



vii. Assume that some external 1 MB SRAM must be fully mapped to the data memory space of the ATxmega128A1U, with the first memory location of the SRAM corresponding to data memory address 0x50 0000. Design, on paper, a hardware expansion for this new system, utilizing the same structure of steps laid out in the relevant pre-lab quiz. Use the same "overall" memory-mapping constraints presented within this quiz, except that which is concerning the usage of the SRAM 3-PORT ALE1 EBI configuration. Hint: you might consider the use of some other EBI configuration.



viii. How would one go about utilizing the SRAM 4PORT NOALE mode? (Note: Looking at the Alternate Pin Functions only ports H, J, and K by default can be used with the EBI system. How can the other port be activated)?

I would need to remap the EBI pins. I would need 4 PORT:

This mode uses four separate ports for EBI communication.

- Port H: Control Signals (RE, WE, CS).
- Port J: 8-bit Data Bus (D7:0).
- Port K: Low Address Byte (A7:0).
- Port F: Middle Address Byte (A15:8).

Also, I won't use ALE, I would use a parallel 8-bit bus (**Port F**) for the middle address byte.

PSEUDOCODE/FLOWCHARTS

SECTION 2

1. Initialize the EBI and IO
   Make EBI control signal in a false state
   Set PORTH bits that we need to use as output
   Set data bits in PORTK to be output

   Set EBI system for SRAM 3-PORT ALE1 mode use EBI_CTRL
   Configure cs2 to a certain size

   Load the EBI_CS2_BASEADDR with the IO start address

2. Set the Z pointer point to IO start address
3. LOOP: load value in Z address and store at r17,
   a. Store the value from r17 back to Z which will trigger the read/write enable clock

   Rjmp LOOP

SECTION 3 part a

1. Define a null byte at the end of the text file
2. Initalize the EBI and I/O pins
   initalize the CS0 and CS2
   load the SRAM_START addres to store in EBI_cs0_BASSADDRESS
3. Initialize the delay :
   Set the PER
   Start the timer
   Create a delay loop that continuously check for timer overflow
   If it's overflow :
   jump to end the timer by clear overflow bits
4. Set Z point to start of the data in the text file
5. Set X point to start of destination SRAM
6. Do a loop : lpm from Z and tst if it's 0 :
   a. It's 0 then rjmp out of the loop
   b. It's not 0 then write to SRAM address point by X
7. Now display these data LOOP:
   a. Read data from SRAM
      if 0 : DONE
      else store that to IO_address for display
      call delay 300ms
      RJMP LOOP

SECTION 3b

2. Call EBI_IO_INIT:

- Set up control pins for EBI (enable address and data signals).
- Configure ports H and K as outputs for address/control lines.

3. Call EBI_INIT:

- Enable 3-port ALE1 EBI mode.
- Configure Chip Select 0 for SRAM access.
- Set the SRAM base address.

4. Infinite Loop (Measurement Loop):

a. Access Address 1 (0xA08000)

- Load X-pointer with address 1.
- Write DATA_VAL_1 (0xAA) to SRAM.
- Read data back from same address.

b. Access Address 2 (0xA08100)

- Load X-pointer with address 2.
- Write DATA_VAL_2 (0x55) to SRAM.
- Read data back from same address.

b. Repeat forever to continuously trigger EBI read/write cycles.

PROGRAM CODE

SECTION 2a

```
;
; lab4_2.asm
;Name: Peter Feng
; class#: 11303
; Pi's Name: Grant
;description: intialize EBI and constantly write from input port to
output port


.include "ATxmega128A1Udef.inc"


;INITIALIZATIONS
 .equ SRAM_START_ADDR = 0xA68000
 .equ IO_START_ADDR = 0xBEE640
 .equ DATA_COUNT = 8




 .org 0x0000
    rjmp MAIN


.org 0x0100
MAIN:
    ; Initialize the stack pointer
    ldi     r16, high(RAMEND)
    sts     CPU_SPH, r16
```

```
    ldi     r16, low(RAMEND)
    sts     CPU_SPL, r16



    ; Initialize the EBI I/O pins and the EBI system
    rcall   EBI_IO_INIT
    rcall   EBI_INIT

     ; To access memory above 0xFFFF, we must use the
    ; Z-pointer along with the RAMPZ register.

    ; 1. Set up the Z-pointer and RAMPZ to point to our I/O address.
    ldi     ZL, Low(IO_START_ADDR)      ; Load low byte of address
    ldi     ZH, high(IO_START_ADDR)      ; Load high byte of address
    ldi     r16, byte3(IO_START_ADDR)      ; Load extended byte of
address
    sts     CPU_RAMPZ, r16                  ; Store it in RAMPZ



MAIN_LOOP:



    ; 2. Read from the external input port (switches).
    ; The 'ld' instruction triggers an EBI read cycle.
    ld    r17, Z       ; Read Z-pointer address into r17

    ; 3. Write to the external output port (LEDs).
```

```
    ;    The 'st' instruction triggers an EBI wriqte cycle.
    st    Z, r17         ; Write from r17 to Z-pointer address

    ; 4. Repeat the process indefinitely.
    rjmp    MAIN_LOOP
/*-------------------------------------------------------------
  ebi_io_init --
  Description:
  Initialize and enable the EBI IO pins for hardware expansion.
-------------------------------------------------------------*/
EBI_IO_INIT:
  ; Preserve the relevant register.
  push r16

  ; Initialize the relevant EBI control signals to be
  ; in a `false` state.
  ;  ALE1 is false (L) by default.
  ; Set active-low EBI signals to false (H) 1s, RE(L) and CS0(L)
  ldi r16, 0b01010011
  sts PORTH_OUTSET, r16



  ldi r16, 0b00000100
  sts PORTH_OUTCLR, r16

  ; Initialize the EBI control signals to be output from
  ; the microcontroller.
```

```
  ;
  ldi r16, 0b01010111
  sts PORTH_DIRSET , r16

  ; Initialize the address signals to be output
  ; from the microcontroller.
  ldi r16, 0xFF
  sts PORTK_DIRSET, r16

  ; Recover the relevant register.
  pop r16

   ; Return from subroutine.
  ret



/*-------------------------------------------------------------
 ebi_init --

 Description:
  Initialize and enable the EBI system for the relevant
  hardware expansion.
-------------------------------------------------------------*/
EBI_INIT:
  push r16

  ; Initialize the EBI system for SRAM 3-PORT ALE1 mode.
```

```
  ; Initialize EBI_CTRL for 3-port (H, J, K) (IFMODE), ALE1 (SRMODE)
  ; Instead of BIT0, can also use EBI_IFMODE_3PORT_gc |
EBI_SRMODE_ALE1_gc
  ldi r16, 0b00000001
  sts EBI_CTRL, r16

  ; Initialize the relevant chip select(s).

  ; Configure chip select CS0
  ; set to 16k  space and turn on SRAM mode
  ; ldi r16, 0b00011101
  ;sts EBI_CS0_CTRLA, r16



;  ldi r16, byte2(SRAM_START_ADDR)

 ; sts EBI_CS0_BASEADDR, r16

  ;ldi r16, byte3(SRAM_START_ADDR)

  ;sts EBI_CS0_BASEADDR+1, r16



  ; Configure chip select CS2
  ; this is just : 0b00001101
  ldi r16, 0b00001101
  sts EBI_CS2_CTRLA, r16
```

```
  ;Load the middle byte (A15:8) of the 3-byte address into a
;  register and store it as the LOW byte of the Base Address,
;  BASEADDRL.  This will store only bits A15:A12 and ignore
;  anything in A11:8 (assumed to be 0).
;Initialize low byte of the EBI CS2 base address (byte2 of address).
   ldi r16, byte2(IO_START_ADDR)
   sts EBI_CS2_BASEADDR, r16

;Load the highest byte (A23:16) of the 3-byte address into a
;  register and store it as the HIGH byte of the Base Address.
;Initialize high byte of the EBI CS2 base address (byte3 of address).
     ldi r16, byte3(IO_START_ADDR)
     sts EBI_CS2_BASEADDR+1, r16

        ; Recover the relevant register.
  pop r16

   ; Return from subroutine.
  ret
```

SECTION 3a

```
;
; lab4_3a.asm
;Name: Peter Feng
; class#: 11303
; Pi's Name: Grant
;Description: store all data forom the text file to SRAM and read from it



.include "ATxmega128A1Udef.inc"

.org 0x0400
DATA_START:
    .include "sram_data_asm.txt"
DATA_END:



;INITIALIZATIONS
.equ SRAM_START_ADDR = 0xA68000
.equ IO_START_ADDR = 0xBEE640


 .org 0x0000
      rjmp MAIN

.org 0x0100
MAIN:
    ; Initialize the stack pointer
    ldi     r16, high(RAMEND)
    sts     CPU_SPH, r16
```

```
    ldi     r16, low(RAMEND)
    sts     CPU_SPL, r16



    ; Initialize the EBI I/O pins and the EBI system
    rcall   EBI_IO_INIT
    rcall   EBI_INIT
      rcall   DELAY_INIT

      ;WRITE DATA FROM PROGRAM MEMORY TO EXTERNAL SRAM
      ; the data is at 0x000400
      ldi ZL, low(DATA_START * 2)  ; data memory is word addressed, multiply by 2
      ldi ZH, high(DATA_START * 2)
      ; so the extend byte is 0;
      clr r16
      sts CPU_RAMPZ, r16

      ; Set X-pointer to the start of the destination SRAM
    ldi     XL, low(SRAM_START_ADDR)
    ldi     XH, high(SRAM_START_ADDR)
    ldi     r16, byte3(SRAM_START_ADDR)
    sts     CPU_RAMPX, r16



WRITE_LOOP:

      lpm r16, Z+  ;load byte from program memory, and move to next address line
      tst r16                 ; test if byte == 0
      breq WRITE_DONE      ; If zero, exit loop
      st X+, r16   ;store the data into SRAM and increment to next address line
```

```
      rjmp WRITE_LOOP


WRITE_DONE:
    st X+, r16    ; put that null into SRAM too

      ; at this point all data has been store inside the SRAM

      ;READ DATA FROM SRAM AND DISPLAY ON I/O PORT WITH DELAY ---
    ; Reset X-pointer to the start of the SRAM to read data back
    ldi     XL, low(SRAM_START_ADDR)
    ldi     XH, high(SRAM_START_ADDR)
    ldi     r16, byte3(SRAM_START_ADDR)
    sts     CPU_RAMPX, r16

      ; Set Z-pointer to the I/O Port address for output
    ldi     ZL, low(IO_START_ADDR)
    ldi     ZH, high(IO_START_ADDR)
    ldi     r16, byte3(IO_START_ADDR)
    sts     CPU_RAMPZ, r16




      DISPLAY_LOOP:
    ld r17, X+           ; Read next byte from SRAM
    tst r17              ; Check if it's NULL (0x00)
    breq DONE            ; Stop when end of data
    st Z, r17            ; Output byte to I/O (LEDs)
    rcall DELAY_300MS
    rjmp DISPLAY_LOOP


DONE:
```

```
    rjmp    DONE        ; Program finished, enter an infinite loop



/*-------------------------------------------------------------
  ebi_io_init --
  Description:
  Initialize and enable the EBI IO pins for hardware expansion.
-------------------------------------------------------------*/
EBI_IO_INIT:
  ; Preserve the relevant register.
  push r16

  ; Initialize the relevant EBI control signals to be
  ; in a `false` state.
  ;   ALE1 is false (L) by default.
  ; Set active-low EBI signals to false (H) 1s, RE(L) and CS0(L)
  ldi r16, 0b01010011
  sts PORTH_OUTSET, r16



  ldi r16, 0b00000100
  sts PORTH_OUTCLR, r16

  ; Initialize the EBI control signals to be output from
  ; the microcontroller.
  ;
  ldi r16, 0b01010111
  sts PORTH_DIRSET , r16

  ; Initialize the address signals to be output
  ; from the microcontroller.
```

```
  ldi r16, 0xFF
  sts PORTK_DIRSET, r16

  ; Recover the relevant register.
  pop r16

   ; Return from subroutine.
  ret



/*-------------------------------------------------------------
 ebi_init --

 Description:
  Initialize and enable the EBI system for the relevant
  hardware expansion.
-------------------------------------------------------------*/
EBI_INIT:
  push r16

  ; Initialize the EBI system for SRAM 3-PORT ALE1 mode.
  ; Initialize EBI_CTRL for 3-port (H, J, K) (IFMODE), ALE1 (SRMODE)
  ; Instead of BIT0, can also use EBI_IFMODE_3PORT_gc | EBI_SRMODE_ALE1_gc
  ldi r16, 0b00000001
  sts EBI_CTRL, r16

  ; Initialize the relevant chip select(s).

   ;Configure chip select CS0
  ; set to 16k  space and turn on SRAM mode
   ldi r16, 0b00011101
```

```
  sts EBI_CS0_CTRLA, r16

 ldi r16, byte2(SRAM_START_ADDR)

 sts EBI_CS0_BASEADDR, r16

  ldi r16, byte3(SRAM_START_ADDR)

  sts EBI_CS0_BASEADDR+1, r16


  ; Configure chip select CS2
  ; this is just : 0b00001101
  ldi r16, 0b00001101
  sts EBI_CS2_CTRLA, r16



  ;Load the middle byte (A15:8) of the 3-byte address into a
;  register and store it as the LOW byte of the Base Address,
;  BASEADDRL.  This will store only bits A15:A12 and ignore
;  anything in A11:8 (assumed to be 0).
;Initialize low byte of the EBI CS2 base address (byte2 of address).
   ldi r16, byte2(IO_START_ADDR)
   sts EBI_CS2_BASEADDR, r16

;Load the highest byte (A23:16) of the 3-byte address into a
;  register and store it as the HIGH byte of the Base Address.
;Initialize high byte of the EBI CS2 base address (byte3 of address).
      ldi r16, byte3(IO_START_ADDR)
      sts EBI_CS2_BASEADDR+1, r16
```

```
        ; Recover the relevant register.
  pop r16

   ; Return from subroutine.
  ret

  DELAY_INIT:
      ; Configure Timer/Counter's Period register for the 300ms delay
      ;  2MHz system clock, choose prescaler = 64
      ; f_timer = f_clk / 64 = 31250Hz
      ; period = T_delay * f_timer = 0.3second * 31250Hz = 9375

      ldi r16, low(9375)
      sts TCC0_PER, r16
      ldi r16, high(9375)
      sts TCC0_PER+1, r16
      ret

  DELAY_300MS:
      ; Start Timer/Conuter C0 with 64 prescaler
      ldi r16, TC_CLKSEL_DIV64_gc
      sts TCC0_CTRLA, r16

; it does nothing but repeatedly checking timer status
 WAIT:
    lds     r16, TCC0_INTFLAGS ; load Interrupt Flags of timer in to r17.
    sbrc    r16, TC0_OVFIF_bp ; if TC0_OVFIF is 0, still counting
    rjmp    WAIT_END          ; otherwise timer has reach and overflow
    rjmp    WAIT
```

```
WAIT_END:
     ; Stop the timer and clear the flag before returning
   ldi     r16, TC_CLKSEL_OFF_gc
   sts     TCC0_CTRLA, r16
   ldi     r16, TC0_OVFIF_bm      ; Load the bit mask for the overflow flag
   sts     TCC0_INTFLAGS, r16      ; Write 1 to the flag to clear it
   ret
```

SECTION 3b

```
;
; lab4_3b.asm
; Name: Peter Feng
; class#: 11303
; Pi's Name: Grant
; Description: Infinitely writes and reads to two SRAM locations
;              to allow for measurement of EBI read/write cycles
;              with a logic analyzer.



.include "ATxmega128A1Udef.inc"



; We need two addresses where the "middle byte" (A15-A8) is different.
; This forces the EBI to toggle the ALE1 signal when switching between them.
; Address 1: 0xA08000 -> Middle byte = 0x80
; Address 2: 0xA08100 -> Middle byte = 0x81
.equ SRAM_ADDR_1 = 0xA08000
.equ SRAM_ADDR_2 = 0xA08100
```

```
; Non-zero 4-bit values to write, making them visible on D3-D0.
; 0xAA = 10101010b
; 0x55 = 01010101b
.equ DATA_VAL_1 = 0xAA
.equ DATA_VAL_2 = 0x55



.org 0x0000
    rjmp MAIN

.org 0x0100
MAIN:
    ; Initialize the stack pointer
    ldi     r16, high(RAMEND)
    sts     CPU_SPH, r16
    ldi     r16, low(RAMEND)
    sts     CPU_SPL, r16

    ; Initialize the EBI I/O pins and the EBI system
    rcall   EBI_IO_INIT
    rcall   EBI_INIT

; This infinite loop will continuously generate write and read cycles
```

```
; to the two specified addresses for capture.
MEASUREMENT_LOOP:
    ; ACCESS ADDRESS 1
    ; Set X-pointer to the first SRAM address
    ldi     XL, low(SRAM_ADDR_1)
    ldi     XH, high(SRAM_ADDR_1)
    ldi     r16, byte3(SRAM_ADDR_1)
    sts     CPU_RAMPX, r16

    ; Write first data value to SRAM
    ldi     r17, DATA_VAL_1
    st      X, r17          ; 1. capture EBI write cycle

    ; Read the data back from SRAM
    ld      r18, X          ;  2. Capture EBI Read cycle


    ; ACCESS ADDRESS 2
    ; Set X-pointer to the second SRAM address.
    ; This address change will trigger the ALE1 signal.
    ldi     XL, low(SRAM_ADDR_2)
    ldi     XH, high(SRAM_ADDR_2)
    ldi     r16, byte3(SRAM_ADDR_2)
    sts     CPU_RAMPX, r16
```

```
    ; Write second data value to SRAM
    ldi     r17, DATA_VAL_2 ;
    st      X, r17          ;  1. capture write cycle

    ; Read the data back from SRAM
    ld      r18, X          ;  2. Capture Read cycle

    ; Repeat forever
    rjmp    MEASUREMENT_LOOP



/*-------------------------------------------------------------
--
 ebi_io_init --
 Description:
 Initialize and enable the EBI IO pins for hardware expansion.
-------------------------------------------------------------
*/
EBI_IO_INIT:
    push    r16
    ldi     r16, 0b01010011
    sts     PORTH_OUTSET, r16
    ldi     r16, 0b00000100
    sts     PORTH_OUTCLR, r16
```

```
    ldi     r16, 0b01010111
    sts     PORTH_DIRSET, r16
    ldi     r16, 0xFF
    sts     PORTK_DIRSET, r16
    pop     r16
    ret


/*-------------------------------------------------------------
--
 ebi_init --
 Description:
 Initialize and enable the EBI system for the relevant
 hardware expansion.
-------------------------------------------------------------
*/
EBI_INIT:
    push    r16
    ; Initialize the EBI system for SRAM 3-PORT ALE1 mode.
    ldi     r16, 0b00000001
    sts     EBI_CTRL, r16

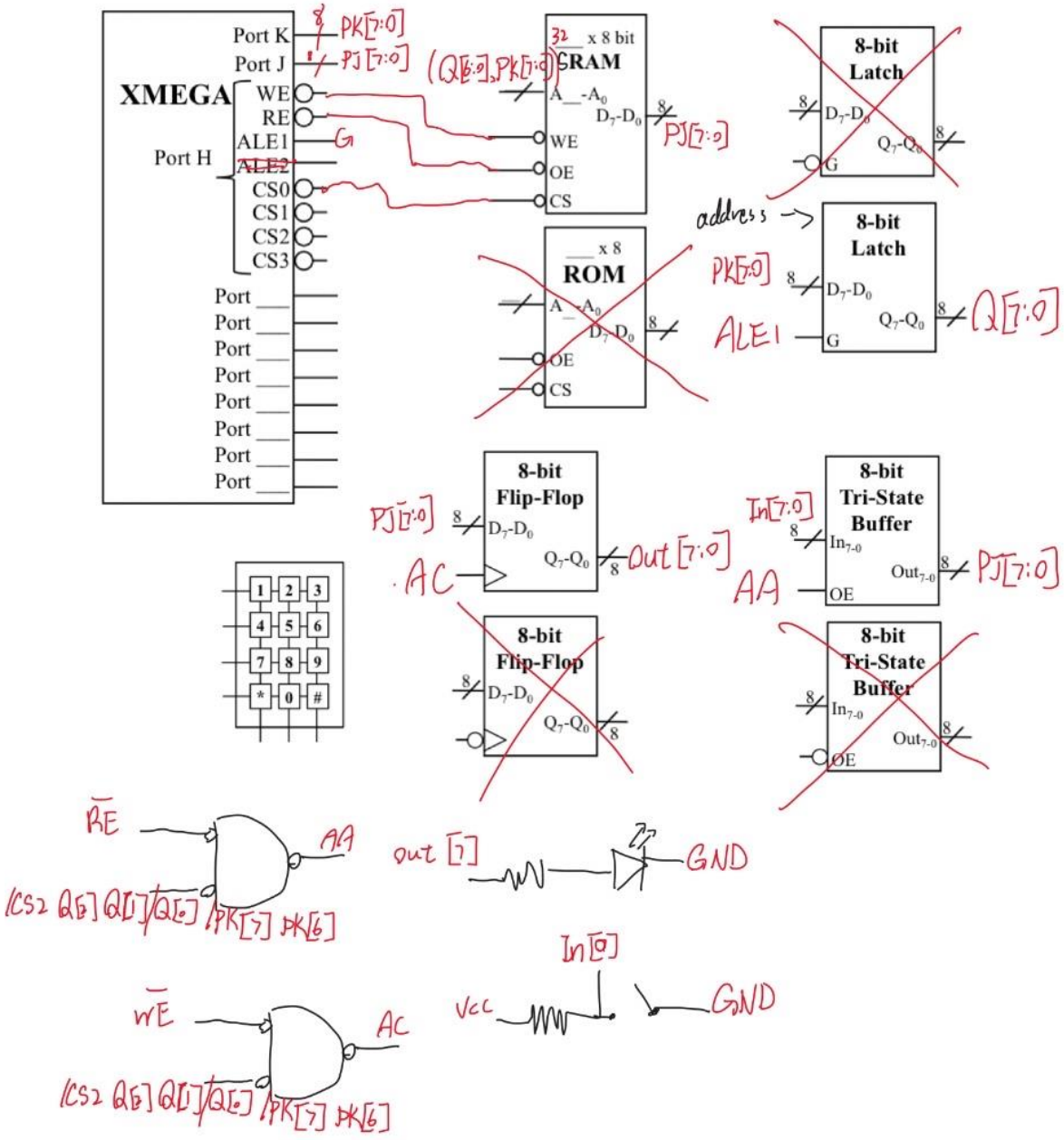
    ; Configure chip select CS0 for SRAM
    ldi     r16, 0b00011101
    sts     EBI_CS0_CTRLA, r16
    ldi     r16, byte2(SRAM_ADDR_1)
```

```
    sts     EBI_CS0_BASEADDR, r16
    ldi     r16, byte3(SRAM_ADDR_1)
    sts     EBI_CS0_BASEADDR+1, r16

    pop     r16
    ret
```

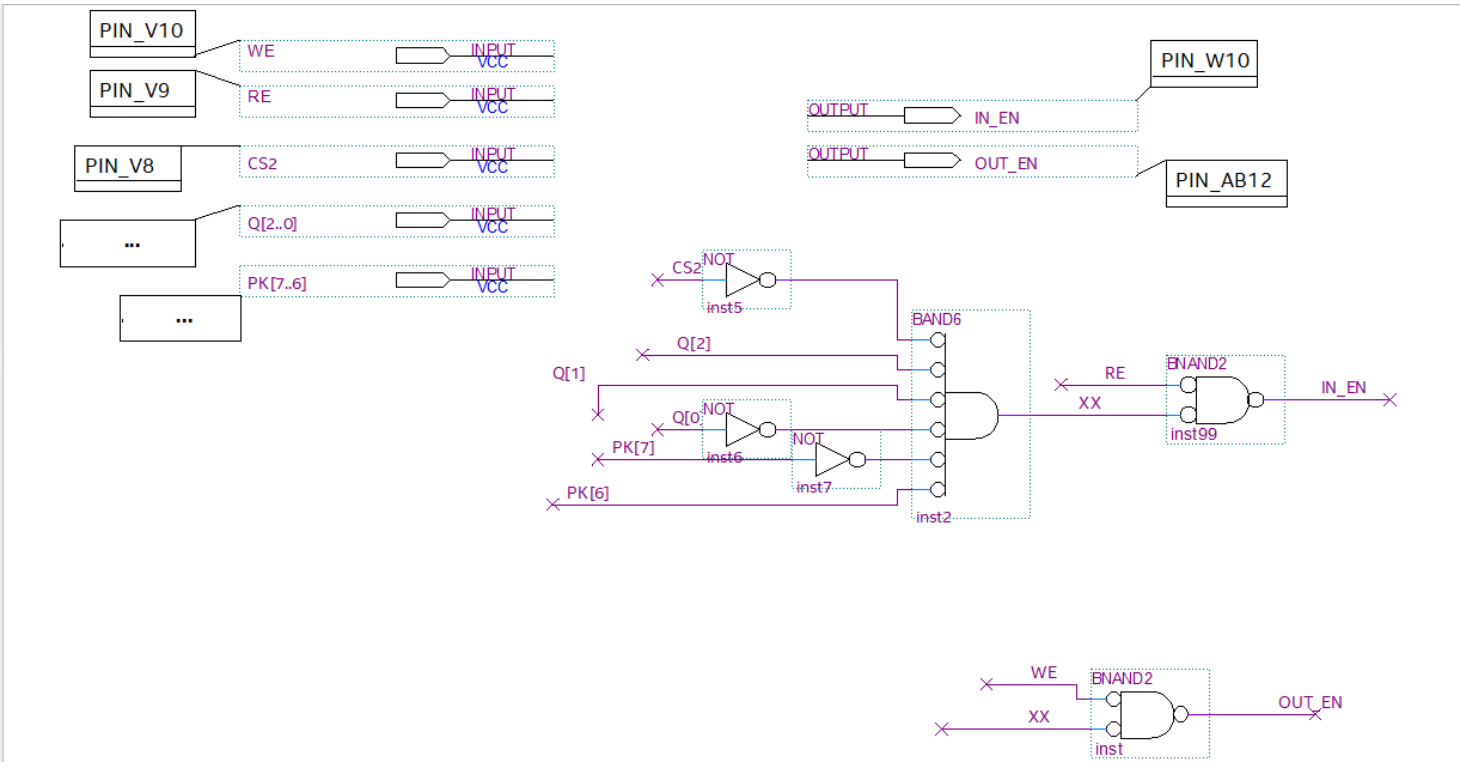
APPENDIX

SECTION 1.3



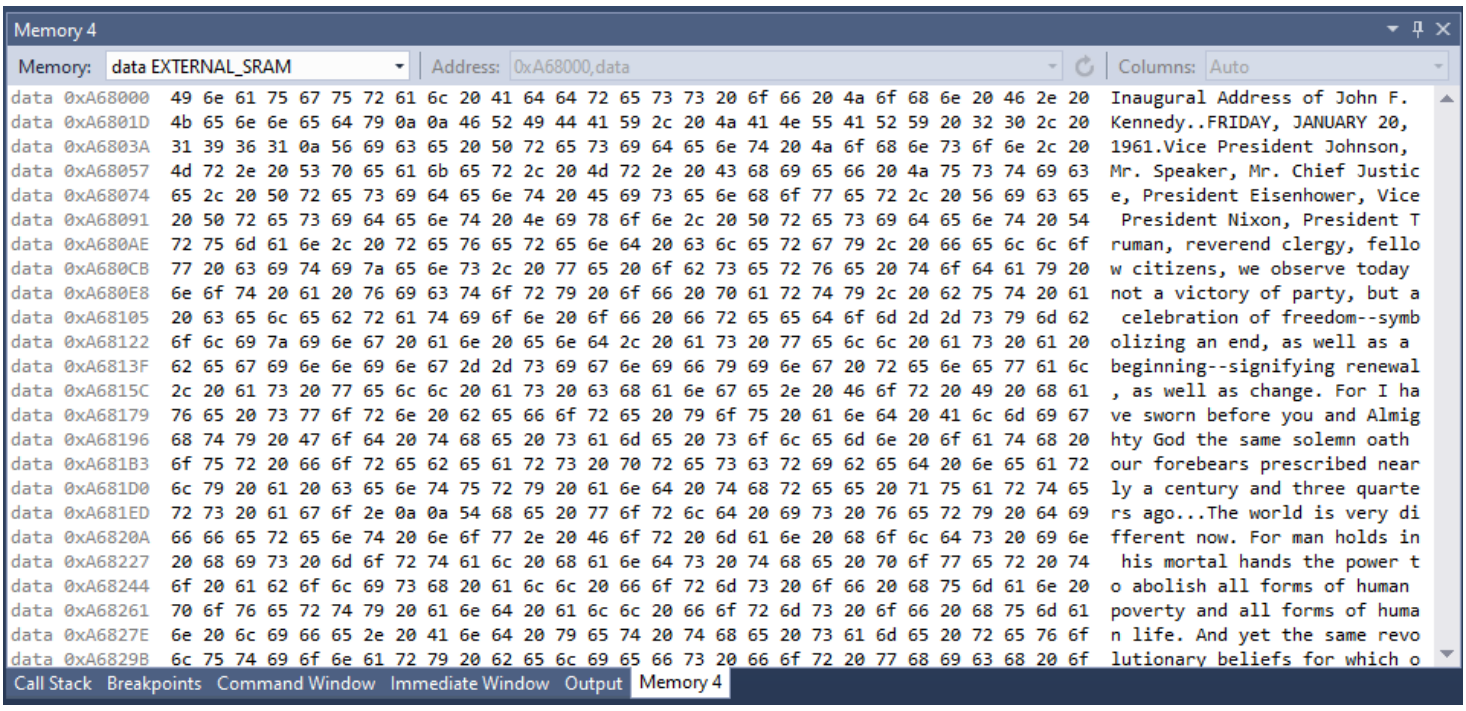
Schematic of the EBI program with more descriptive labels

SECTION 2a



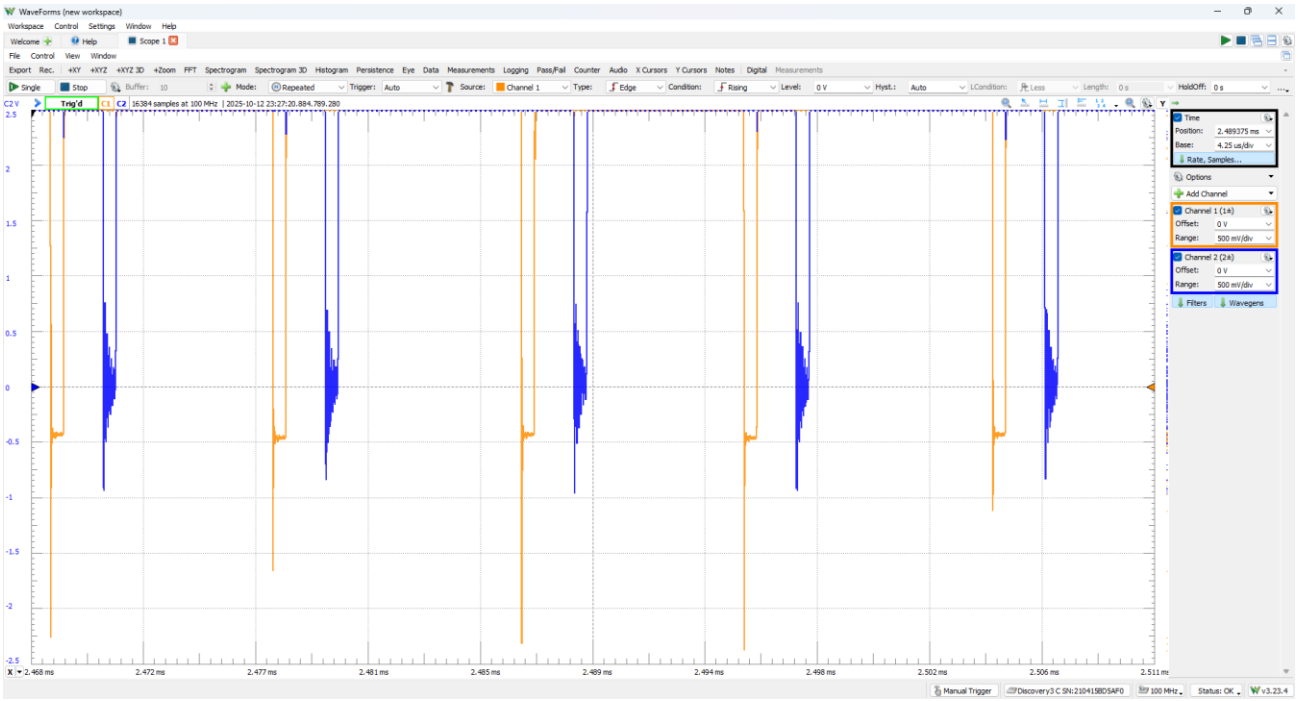
BDF screenshot of 2 BNAND gate PLD logic for input and output enable

SECTION 3a



External SRAM after data written into it

SECTION 3b

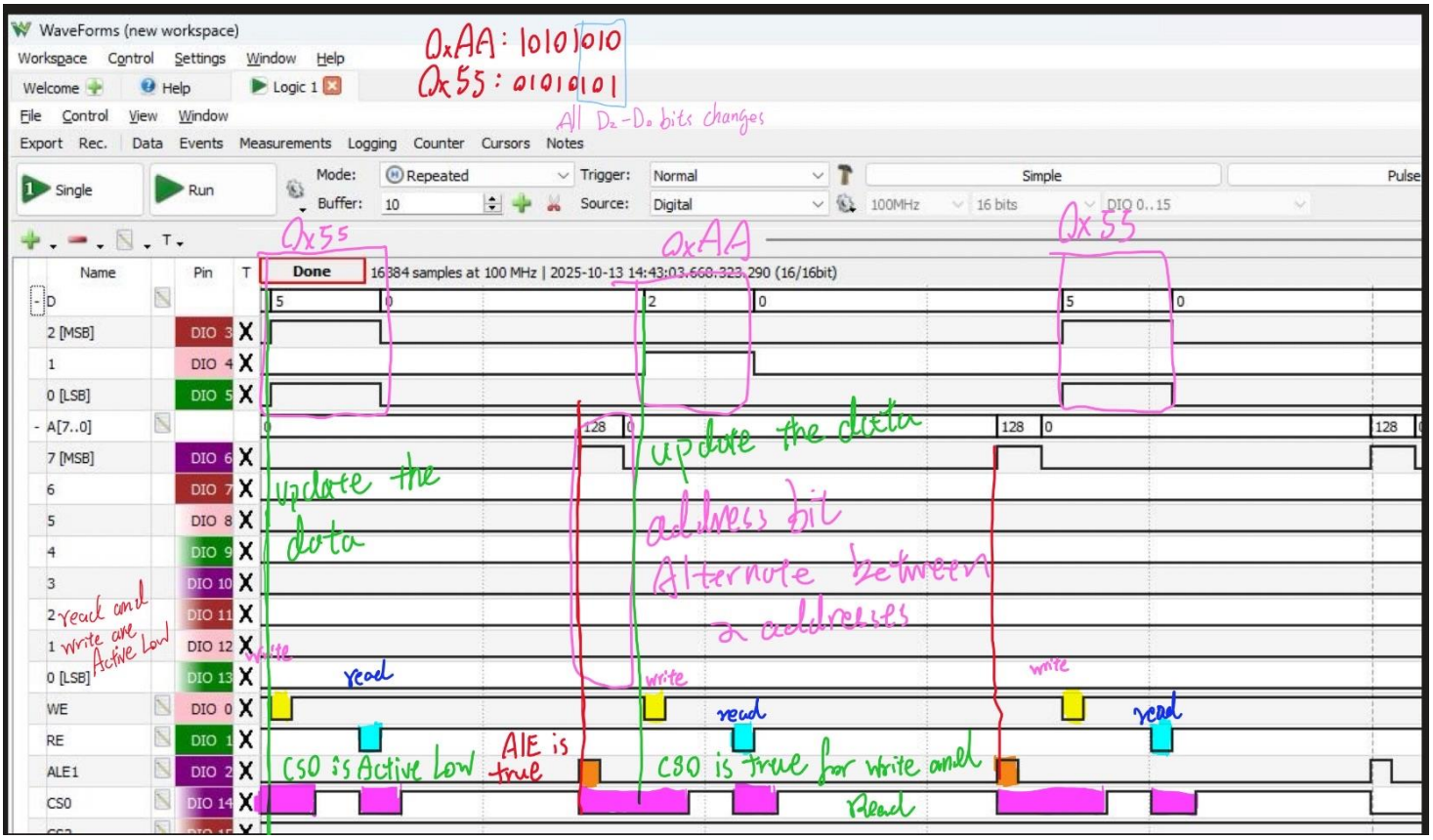


The Orange pin is plug into OUTPUT on the D flip flop, The blue pin is plug into INPUT on the tri-state buffer.

They have the same frequency and around 0.008 ms period.



This pin is plug into the LED, this shows the frequency of the output.



SECTION 3b The annotated Logic Analyzer